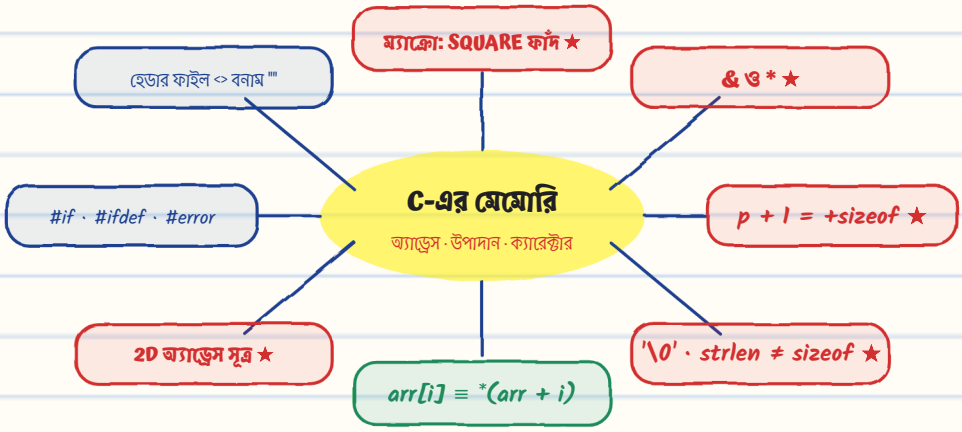
C-তে অ্যাড্রেস, অ্যারে ও স্ট্রিং

হেডার ফাইল · প্রিপ্রসেসর · ম্যাক্রো · পয়েন্টার · 1D/2D অ্যারে · স্ট্রিং: ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো লেসন এক পাতায়: লাল = বেশি আসে, সবুজ = যে নিয়মের ওপর বাকি সব দাঁড়িয়ে

➜ সিলেবাসের যে অংশ

Structured Programming Language সিলেবাসে আছে: "*Header files; Preprocessor; Pointers and arrays; Strings; Multidimensional array*"। ডেটা টাইপ, অপারেটর ও কন্ট্রোল ফ্লো, এবং ফাংশন, স্কোপ ও স্টোরেজ ক্লাস আগের লেসনে; স্ট্রাকচার, ইউনিয়ন ও *enum*, তারপর *I/O*, ফাইল ও `malloc` পরের লেসনে।

হাই-ইল্ড তালিকা

① `<>` সিস্টেম ডিরেক্টরি; `""` আগে নিজের ফোল্ডার
② প্রিপ্রসেসর = কম্পাইলের আগে টেক্সটের কাজ
③ `x*x` দিলে `SQUARE(2+3)` → *11*
④ `&` অ্যাড্রেস; `*` ওই অ্যাড্রেসের মান
⑤ `p + 1` সরে `sizeof(*p)` বাইট
⑥ `arr[i] ≡ *(arr+i) ≡ i[arr]`
⑦ `sizeof(arr) ≠ sizeof(p)`
⑧ *C* রো-মেজর
⑨ স্ট্রিং শেষ হয় `'\0'`-এ *(ASCII 0)*
⑩ `gets()` *C11*-এ বাদ → `fgets()`

সংকেত হলুদ = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

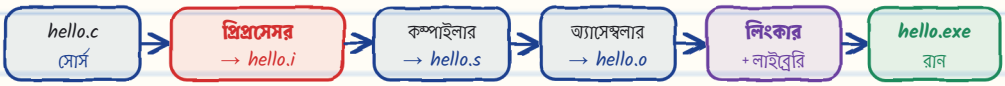
➜ সূচিপত্র

✓ হেডার ফাইল ও বিল্ড পাইপলাইন ... পৃ. ২
✓ প্রিপ্রসেসর ডিরেক্টিভ ও ম্যাক্রো ... পৃ. ৩
✓ পয়েন্টার: `&`, `*`, প্রকার, *const* ... পৃ. ৪
✓ পয়েন্টার অ্যারিথমেটিক ... পৃ. ৫
✓ *1D* অ্যারে ও অ্যারে–পয়েন্টার সম্পর্ক ... পৃ. ৬
✓ *2D* অ্যারে ও অ্যাড্রেস সূত্র ... পৃ. ৭
✓ স্ট্রিং ও `'\0'` ... পৃ. ৮
✓ *string.h* ও স্ট্রিং ইনপুট ... পৃ. ৯
✓ রাতের পরীক্ষার পাতা ... পৃ. ১০
✓ অনুশীলনী *MCQ* + লিখিত ... পৃ. ১১

১ হেডার ফাইল ও বিল্ড পাইপলাইন

MCQ-র প্রিয় ★

সংজ্ঞা হেডার ফাইল (Header File) (.h) একাধিক সোর্স ফাইলের ভাগ করা **ডিক্লারেশন** রাখে: ফাংশন প্রোটোটাইপ, ম্যাক্রো, কনস্ট্যান্ট ও টাইপ সংজ্ঞা (typedef, struct)। #include লাইনটির জায়গায় ফাইলের পুরো টেক্সট বসিয়ে দেয় — **কম্পাইলের আগেই**। হেডারে কোনো কম্পাইলড কোড থাকে না: printf, sqrt ... -এর বডি থাকে লাইব্রেরিতে, যা জোড়ে **লিংকার (Linker)**।



হেডার বসে প্রিপ্রসেসর ধাপে; লাইব্রেরির কোড জোড়ে লিংকার ধাপে

gcc -E প্রিপ্রসেসিংয়ের পরে থামে, -S কম্পাইলের পরে, -c অ্যাসেম্বলের পরে

হেডার	কী দেয় · যে নামগুলো চিনতে হবে
<stdio.h>	স্ট্যান্ডার্ড I/O: printf, scanf, puts, fgets, getchar, fopen, FILE, EOF
<conio.h>	কনসোল I/O, **স্ট্যান্ডার্ড C নয়** (Turbo C/DOS, MSVC): clrscr, getch, getche
<math.h>	double-এ গণিত: sqrt, pow, fabs, ceil, floor, sin, log (gcc: -lm দিয়ে লিংক)
<string.h>	স্ট্রিং ও মেমোরি ব্লক: strlen, strcpy, strcat, strcmp, strstr, memcpy
<stdlib.h>	সাধারণ ইউটিলিটি: malloc, calloc, free, atoi, rand, exit, abs, qsort
<ctype.h>	ক্যারেক্টার পরীক্ষা ও কেস: isalpha, isdigit, isspace, toupper, tolower
<time.h>	তারিখ ও সময়: time, clock, difftime, CLOCKS_PER_SEC
<limits.h>	ইন্টিজারের সীমা: INT_MAX, INT_MIN, CHAR_BIT, LONG_MAX
<float.h>	ফ্লোটিং-পয়েন্টের সীমা: FLT_MAX, DBL_MAX, FLT_EPSILON, FLT_DIG
<stddef.h> · <stdbool.h>	NULL, size_t, ptrdiff_t · bool, true, false (C99)

```c
/* myheader.h */
#ifndef MYHEADER_H      /* include guard */
#define MYHEADER_H
int add(int a, int b); /* prototype */
#define MAX 100
#endif
/* main.c */
#include <stdio.h>      /* system dirs */
#include "myheader.h"   /* own dir 1st */
```

<...> বনাম "..." <stdio.h> খোঁজা হয় কেবল স্ট্যান্ডার্ড (সিস্টেম) ইনক্লুড ডিরেক্টরিতে — লাইব্রেরি হেডারের জন্য। "my.h" খোঁজা হয় আগে নিজের ফোল্ডারে, তারপর সিস্টেম ডিরেক্টরিতে — নিজের হেডারের জন্য। ইনক্লুড গার্ড (Include Guard) দ্বিতীয় কপি আটকায়, তাই "redefinition" এরর হয় না; #pragma once একই কাজ করে (স্ট্যান্ডার্ড নয়, তবে প্রায় সব কম্পাইলারে চলে)।

মনে রাখার কৌশল "**অ্যাঙ্গেল ব্র্যাকেট = লাইব্রেরির তাক; কোটেশন = নিজের টেবিল**, আগে টেবিল দেখো, তারপর তাক।"

পরীক্ষার ফাঁদ হেডার তার লাইব্রেরি লিংক করে **না**: <math.h> থাকলেও gcc-তে -lm লাগে, নইলে "*undefined reference*"। conio.h, clrscr(), getch() ISO C নয়।

সমাধানকৃত উদাহরণ ✍ কোন হেডার?

INT_MAX → *limits.h* · toupper → *ctype.h* · malloc → *stdlib.h* · fabs → *math.h* কিন্তু abs → *stdlib.h*।

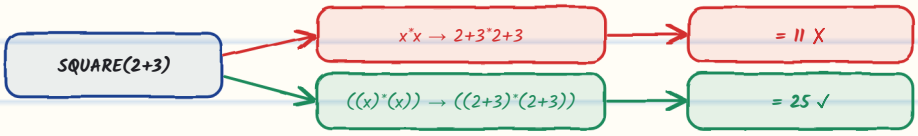
২ প্রিপ্রসেসর ডিরেক্টিভ ও ম্যাক্রো আউটপুট প্রশ্ন ★

সংজ্ঞা প্রিপ্রসেসর *(Preprocessor)* চলে **কম্পাইলারের আগে** এবং # দিয়ে শুরু হওয়া প্রতিটি লাইন **(ডিরেক্টিভ, *Directive*)** সামলায়। এর কাজ পুরোপুরি টেক্সটের — ফাইল যুক্ত করা, ম্যাক্রো *(Macro)* প্রতিস্থাপন, শর্তসাপেক্ষ কম্পাইলেশন — কমেন্ট সরানোর পরে। ডিরেক্টিভ লাইনের শেষে শেষ হয়: **সেমিকোলন নেই** (লম্বা হলে \ দিয়ে পরের লাইনে)।

ডিরেক্টিভ	কাজ
#include	ফাইলের টেক্সট যুক্ত করে
#define	কনস্ট্যান্ট বা ম্যাক্রো: #define PI 3.1416
#undef	ম্যাক্রোর সংজ্ঞা মুছে দেয়
#ifdef · #ifndef	সংজ্ঞায়িত হলে · না হলে কম্পাইল
#if #elif #else #endif	ধ্রুবক ইন্টিজার শর্তে কম্পাইল; defined(X) চলে
#error msg	বার্তা দিয়ে কম্পাইল থামায়
#pragma · #line	কম্পাইলার-নির্দিষ্ট নির্দেশ (once, pack) · লাইন নম্বর বদল

```
#define DEBUG 1
#define VERSION 2
#ifdef DEBUG
  printf("Debug mode ON\n");
#endif
#if VERSION >= 2
  newApi();
#elif VERSION == 1
  oldApi();
#else
  #error "unknown VERSION"
#endif
#undef DEBUG /* gone from here */
```

পূর্বনির্ধারিত ম্যাক্রো *(Predefined Macro)*: __FILE__ ফাইলের নাম (স্ট্রিং) · __LINE__ লাইন নম্বর *(int)* · __DATE__ "Oct 14 2026" · __TIME__ "hh:mm:ss" · __STDC__ মানসম্মত কম্পাইলারে 1 · __STDC_VERSION__ 199901L *(C99)*, 201112L *(C11)*।



ম্যাক্রো টেক্সট হিসেবে বসে; তারপর স্বাভাবিক অগ্রাধিকার খাটে

বিষয়	ম্যাক্রো	ফাংশন
কখন	প্রিপ্রসেসিং: টেক্সট	রান টাইম: কল
টাইপ চেক	**নেই**	আছে (প্রোটোটাইপ)
আর্গুমেন্ট	প্রতি ব্যবহারে আবার মূল্যায়ন	একবার মূল্যায়ন
গতি · আকার	কলের খরচ নেই · কোড বাড়ে	কলের খরচ · এক কপি
অ্যাড্রেস	নেই	ফাংশন পয়েন্টার

ও ## # স্ট্রিং বানায়: #define STR(x) #x, STR(Dhaka) → "Dhaka"। ## টোকেন জোড়ে: #define CAT(a,b) a##b, CAT(var,1) → var1। ফাংশন-সদৃশ ম্যাক্রোর চেয়ে নিরাপদ: inline ফাংশন *(C99)*।

মনে রাখার কৌশল "**ভেতরে ব্র্যাকেট, বাইরেও ব্র্যাকেট**": প্রতিটি প্যারামিটার ও পুরো বডি মুড়ে দাও — ((x)*(x))।

পরীক্ষার ফাঁদ #define PI 3.14; দিলে ;-ও বসে যায়। SQUARE(i++) *i*-কে দুবার বাড়ায়: অনির্ধারিত। #define SQ (x)-এ ফাঁকা থাকলে তা অবজেক্ট-সদৃশ।

সমাধানকৃত উদাহরণ ✍

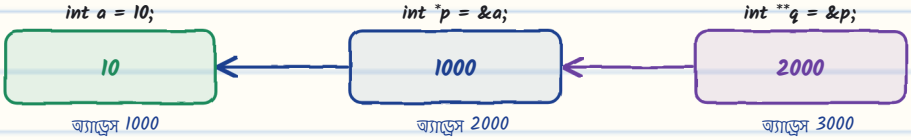
আউটপুট

#define SQ(x) (x)*(x): 100/SQ(5) → 100/(5)*(5) = 20×5 = 100, 4 নয়। পুরো ব্র্যাকেটে: 4।

৩ পয়েন্টার: &, *, প্রকার ও const

সবচেয়ে বেশি আসে ★

সংজ্ঞা পয়েন্টার (Pointer) এমন একটি ভেরিয়েবল যা অন্য অবজেক্টের **মেমোরি অ্যাড্রেস** রাখে: `type *name;` । `&` (**অ্যাড্রেস-অফ**) অ্যাড্রেস দেয়; `*` (**ইনডিরেকশন / ডিরেফারেন্স**, "ওই অ্যাড্রেসের মান") সেই অ্যাড্রেসের অবজেক্ট দেয়। পয়েন্টারের নিজের আকার মেশিন ঠিক করে — 32-বিটে 4 বাইট, 64-বিটে 8 — যেটাকেই দেখাক; তার টাইপ ঠিক করে `*` কত বাইট পড়বে আর `+1` কতদূর সরবে।



*p = 10 · **q = 10 · *q = p = 1000 · &p = 2000

পয়েন্টারের বাক্সে থাকে অন্য বাক্সের অ্যাড্রেস

```
int a = 10;
int *p;          /* declare      */
p = &a;          /* address of a */
printf("%d", *p); /* 10          */
*p = 20;         /* a is now 20  */
int **q = &p;    /* ptr to ptr   */
printf("%d", **q);/* 20          */
int *x, y;       /* x ptr, y int! */
```

এক্সপ্রেশন	মান
a · &a	20 · 1000
p · *p · &p	1000 · 20 · 2000
*&a (পরস্পর কাটে)	20
q · *q · **q	2000 · 1000 · 20
printf("%p", (void *)p)	1000 ছাপে

প্রকার	কী	বিপদ ও প্রতিকার
NULL	`int *p = NULL;` কোথাও দেখায় না	ডিরেফারেন্স → ক্র্যাশ; আগে `p != NULL` দেখো
void (জেনেরিক)	যেকোনো অবজেক্টের অ্যাড্রেস রাখে; `malloc` এটাই দেয়	কাস্ট ছাড়া `*v` বা `v+1` নয়: `*(int *)v`
ওয়াইল্ড (Wild)	কখনো ইনিশিয়ালাইজ হয়নি: আবর্জনা অ্যাড্রেস	`NULL` বা বৈধ অ্যাড্রেস দাও
ড্যাংলিং (Dangling)	অবজেক্ট আর নেই: `free(p)`-এর পরে, বা লোকালের অ্যাড্রেস ফেরত দিলে	`free`-এর পরে `p = NULL;`

ডিক্লারেশন	`*p=5`	`p++`
`const int *p` = `int const *p`	✗	✓
`int *const p = &a;`	✓	✗
`const int *const p`	✗	✗

দেখতে একরকম `int *p[3]`: ৩টি পয়েন্টারের অ্যারে। `int (*p)[3]`: ৩টি *int*-এর অ্যারের দিকে **একটি** পয়েন্টার। `int (*fp)(int)`: ফাংশন পয়েন্টার। ফাংশনে `&x` পাঠানো: আগের লেসন।

মনে রাখার কৌশল "**`&` জিজ্ঞেস করে কোথায়; `*` জিজ্ঞেস করে সেখানে কী।**" ডান থেকে বাঁয়ে পড়ো: `int *const p` = "*p* হলো *int*-এর *const* পয়েন্টার"।

পরীক্ষার ফাঁদ `int *x, y;`-এ কেবল **x** পয়েন্টার: `*` নামের সঙ্গে লাগে। *NULL* বা ওয়াইল্ড ডিরেফারেন্স কম্পাইল হয়, রান টাইমে ভাঙে।

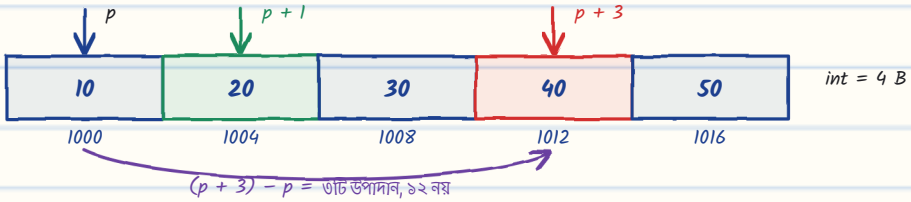
সমাধানকৃত উদাহরণ ✍ ট্রেস

`int a = 5, *p = &a, **q = &p; **q += *p;` → দুটোই *a*, তাই *a* = 5 + 5 = 10।

৪ পয়েন্টার অ্যারিথমেটিক

সংখ্যাভিত্তিক MCQ★

সংজ্ঞা পয়েন্টারে ইন্টিজার *n* যোগ করলে তা ***n* টি উপাদান** সরে, *n* বাইট নয়: `address(p + n) = address(p) + n × sizeof(*p)`। একই অ্যারের দুটি পয়েন্টার বিয়োগ করলে মাঝের **উপাদানের সংখ্যা** পাওয়া যায় (টাইপ `ptrdiff_t`)। ফল নির্ধারিত কেবল একটি অ্যারের ভেতরে, আর **শেষের ঠিক পরের** অবস্থানে।



`int a[5] = {10,20,30,40,50}; int *p = a;` — প্রতি ধাপে ৪ বাইট লাফ

পয়েন্টার টাইপ	sizeof(*p)	1000 থেকে p+1
char *	1	1001
short *	2	1002
int *, float *	4	1004
double *	8	1008
int ** (64-বিট)	8	1008
12 বাইটের struct *	12	1012

অপারেশন	চলে?	ফল
p + n, p − n, p++, p--	✓	n উপাদান সরে
p2 − p1	✓	মাঝের উপাদান
p1 < p2, p == NULL	✓	অবস্থান তুলনা
p1 + p2	✗	কম্পাইল এরর
p * 2, p / 2, p % 2	✗	কম্পাইল এরর
void * অ্যারিথমেটিক	✗	ISO C নয় (gcc: 1 B)

```
int a[] = {10, 20, 30, 40};
int *p = a;
int x = *p++;   /* x=10, p->a[1] */
int y = (*p)++; /* y=20, a[1]=21 */
int z = *++p;   /* z=30, p->a[2] */
printf("%d %d %d %d", x, y, z, a[1]);
                /* 10 20 30 21   */
int *q = &a[3];
printf("%d", (int)(q - a)); /* 3 */
```

এক্সপ্রেশন	যা বদলায়	ব্যবহৃত মান
*p++ = *(p++)	**p**, পরে	পুরোনো *p
*++p	**p**, আগে	নতুন *p
(*p)++	**মান**, পরে	পুরোনো মান
++*p	**মান**, আগে	নতুন মান

পয়েন্টার দিয়ে অ্যারে ঘোরা `for (p = a; p < a + n; p++) sum += *p;` — `a + n` (শেষের ঠিক পরের অবস্থান)-এর সঙ্গে তুলনা বৈধ, কিন্তু `*(a + n)` পড়া বৈধ নয়। **ভিন্ন** অ্যারের পয়েন্টার বিয়োগ বা `<` দিয়ে তুলনা করা যায় না; `==` সবসময় চলে। পয়েন্টারের পার্থক্য ছাপতে `%td`।

মনে রাখার কৌশল "**টাইপই মাপার স্কেল।**" পয়েন্টার গোনে উপাদানে; এক ধাপ কত লম্বা তা বলে তার টাইপ।

পরীক্ষার ফাঁদ `*p++` সরায় **পয়েন্টার**, মান নয়: পোস্টফিক্স `++`-এর বাঁধন `*`-এর চেয়ে শক্ত। ১২ বাইট দূরের দুটি `int` পয়েন্টারের পার্থক্য **3**।

সমাধানকৃত উদাহরণ ✍

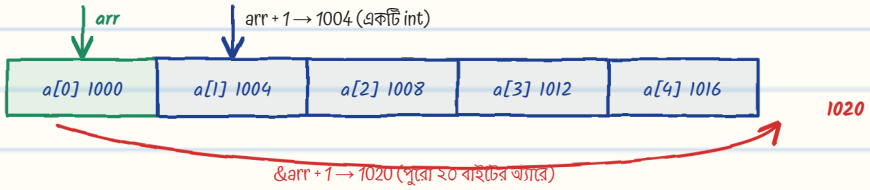
অ্যাড্রেস

`double *d`-এ 2000: `d + 5` = 2000 + 5×8 = **2040**।

`char *` হলে: 2005।

৫ 1D অ্যারে ও অ্যারে–পয়েন্টার সম্পর্ক সবচেয়ে হাই-ইল্ড ★

সংজ্ঞা অ্যারে (Array) হলো **একই টাইপের** উপাদানের নির্দিষ্ট আকারের সংগ্রহ, যা **পরপর (contiguous)** মেমোরিতে থাকে, ইনডেক্স `0 … n-1`; `int arr[5]` নেয় `5 × 4 = 20` বাইট। C **সীমা যাচাই করে না**: `arr[5]` কম্পাইল হয়, কিন্তু অনির্ধারিত আচরণ (Undefined Behaviour)। বেশিরভাগ এক্সপ্রেশনে নামটি ০ নম্বর উপাদানের পয়েন্টারে রূপান্তরিত (Decay) হয়, তাই `arr[i] ≡ *(arr + i)` ≡ `*(i + arr) ≡ i[arr]`।



arr ও &arr একই সংখ্যা ছাপে; তবে টাইপ ও ধাপ আলাদা

এক্সপ্রেশন	টাইপ	মান
`arr`, `&arr[0]`	`int *`	1000
`arr + 1`	`int *`	1004
`&arr`	`int (*)[5]`	1000 (একই!)
`&arr + 1`	`int (*)[5]`	1020
`sizeof(arr)`	`size_t`	20
`sizeof arr / sizeof arr[0]`	`size_t`	5

ডিক্লারেশন	বিষয়বস্তু
`int a[5]={1,2};`	1 2 0 0 0
`int a[]={1,2,3};`	আকার ধরা হয় 3
`int a[5]={0};`	সব শূন্য
`int a[5];` লোকাল · গ্লোবাল	আবর্জনা · সব শূন্য
`int a[5]={[2]=7};`	0 0 7 0 0 (C99)
`int a[n];`	VLA: C99; C11-এ ঐচ্ছিক

```
int sum(int a[], int n) { /* int *a */
    int s = 0;
    for (int i = 0; i < n; i++)
        s += a[i];        /* *(a + i) */
    return s;  /* sizeof(a) = 8 here */
}
int arr[5] = {1, 2, 3, 4, 5};
int n = sizeof arr / sizeof arr[0];
sum(arr, n);   /* passes &arr[0] */
```

কখন নাম রূপান্তরিত হয় না ① `sizeof`-এর ভেতরে → পুরো অ্যারের আকার; ② `&`-এর সঙ্গে → পুরো অ্যারের পয়েন্টার; ③ `char` অ্যারেকে ইনিশিয়ালাইজ করা স্ট্রিং লিটারাল। নামটি **পরিবর্তনযোগ্য lvalue নয়**: `arr++`, `arr = p` চলে না; `p++` চলে। ফাংশন পায় কেবল অ্যাড্রেস, তাই `a[i]`-এর পরিবর্তন কলারের অ্যারেতে পৌঁছায়।

মনে রাখার কৌশল "**অ্যারের নাম = ০ নম্বর উপাদানে পোঁতা স্থির সাইনবোর্ড; পয়েন্টার = হাতে বয়ে বেড়ানো সাইনবোর্ড।**"

পরীক্ষার ফাঁদ অ্যারে প্যারামিটার আসলে পয়েন্টার: ফাংশনের ভেতরে `sizeof(a)` দেয় 8 (বা 4), অ্যারের আকার নয়। `n` সবসময় পাঠাও।

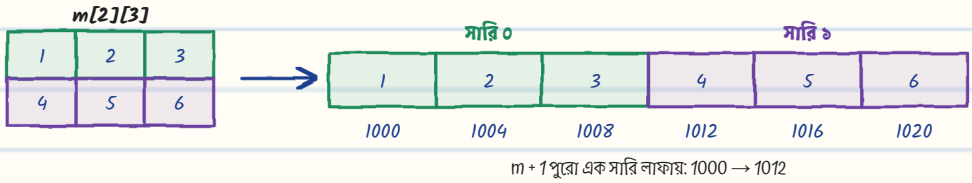
সমাধানকৃত উদাহরণ ✍

```
int a[5]={1,2,3,4,5};
int *p=(int *)(&a+1);
printf("%d",*(p-1)); →
```

আউটপুট 5

৬ 2D অ্যারে ও অ্যাড্রেস সূত্র লিখিত + সংখ্যাভিত্তিক ★

সংজ্ঞা `int m[R][C]` হলো *R* টি সারির অ্যারে, প্রতিটি সারি *C* টি *int*-এর অ্যারে। মেমোরি একটি সরল রেখা, আর *C* উপাদানগুলো রাখে **রো-মেজর ক্রমে (Row-major Order)**: আগে পুরো সারি ০, তারপর সারি ১, … কেবল **প্রথম** (সারির) আকার বাদ দেওয়া যায় — `int m[][3] = {…};`, `void f(int m[][3], int rows)` — কারণ প্রতিটি সারি কোথায় শুরু তা বের করতে **কলামের আকার লাগবেই**।



m + 1 পুরো এক সারি লাফায়: 1000 → 1012

গ্রিড আমাদের ভাবনা; রেখা হলো মেমোরিতে রাখার ধরন

```
int m[2][3] = { {1, 2, 3},
                {4, 5, 6} };
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++)
        printf("%d ", m[i][j]);
    printf("\n");
}            /* 1 2 3
                4 5 6 */
int (*row)[3] = m; /* ptr to a row */
```

এক্সপ্রেশন	অর্থ	মান
`m`	→ সারি ০	1000
`m + 1`	→ সারি ১ (+12 B)	1012
`*m + 1`	`&m[0][1]` (+4 B)	1004
`*(m + 1) + 2`	`&m[1][2]`	1020
`*(*(m + 1) + 2)`	`m[1][2]`	6
`sizeof m · m[0]`	পুরো · এক সারি	24 · 12

অ্যাড্রেস সূত্র *M* = সারি, *N* = কলাম, *W* = প্রতি উপাদানের বাইট, *B* = বেস অ্যাড্রেস, L_r, L_c = নিম্নসীমা (*C*-তে দুটোই ০)।

রো-মেজর: $LOC(A[i][j]) = B + W \times [N \times (i - L_r) + (j - L_c)]$

কলাম-মেজর: $LOC(A[i][j]) = B + W \times [M \times (j - L_c) + (i - L_r)]$

C, C++, Pascal ও *NumPy* (ডিফল্ট) রো-মেজর; *Fortran, MATLAB* ও *R* কলাম-মেজর।

সমাধানকৃত উদাহরণ ✍ A[1..10][1..15], W = 2, B = 100: A[4][8] কোথায়? *M = 10, N = 15*। **রো-মেজর:** $100 + 2 \times [15 \times 3 + 7] = 100 + 2 \times 52 =$ **204**। **কলাম-মেজর:** $100 + 2 \times [10 \times 7 + 3] = 100 + 2 \times 73 =$ **246**। *C*-তে `int A[5][4]` 1000-এ: `&A[3][2]` = $1000 + 4 \times (3 \times 4 + 2) = 1056$।

সমাধানকৃত উদাহরণ ✍ সারির পয়েন্টার `int a[2][3] = {{1,2,3},{4,5,6}}; int (*r)[3] = a;` → `r[1][1] = 5`, `*(*(r + 1)) =` **4** (সারি ১-এর প্রথম উপাদান), `*(*r + 2) = 3`। `r + 1` ১২ বাইট সরে, কারণ `*r` হলো ৩টি *int*-এর পুরো একটি সারি।

মনে রাখার কৌশল **"রো-মেজর বই পড়ার মতো"**: এক লাইন শেষ, তারপর পরের লাইন। কলাম-মেজর পত্রিকার কলাম পড়ার মতো।

পরীক্ষার ফাঁদ `int m[][] = {…};` এরর। `m + 1` এক সারি লাফায়, `*m + 1` এক উপাদান। সীমা *0* না *1* থেকে শুরু, দেখে নাও।

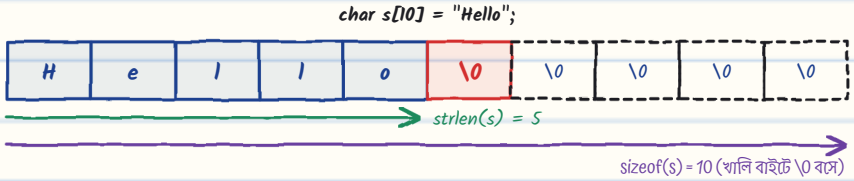
সমাধানকৃত উদাহরণ ✍ আকার

`int a[2][3][4]`-এ ২৪টি *int* = **96 বাইট**; `a[1]` হলো 48 বাইটের একটি 3×4 ব্লক।

৭ স্ট্রিং ও নাল ক্যারেক্টার

MCQ + আউটপুট ★

সংজ্ঞা C-তে আলাদা স্ট্রিং টাইপ নেই: স্ট্রিং (String) হলো এমন char অ্যারে যার শেষে থাকে নাল ক্যারেক্টার (Null Character) '\0' (ASCII মান 0)। প্রতিটি স্ট্রিং ফাংশন — strlen, printf("%s"), strcpy — '\0' না পাওয়া পর্যন্ত এগোয়। তাই লিটারাল "Hello" নেয় ৬ বাইট: ৫টি অক্ষর + কম্পাইলারের যোগ করা টার্মিনেটর।



strlen থামে প্রথম '\0'-এ; sizeof গোনে অ্যারের সব বাইট

ডিক্লারেশন	বাইট	মন্তব্য
char s1[] = {'H','i','\0'};	3	হাতে '\0'
char s2[] = "Hello";	6	'\0' যোগ হয়
char s3[10] = "Hello";	10	বাকি '\0'
char s4[5] = "Hello";	5	'\0' নেই

প্রচলিত ভুল ধারণা টার্মিনেটর হলো '\0', মান 0 — ' ' (স্পেস, 32), '0' (অঙ্ক, 48) বা '\n' (10) নয়। NULL একটি পয়েন্টার; '\0' একটি ক্যারেক্টার।

বিষয়	char s[] = "Dhaka";	char *p = "Dhaka";
এটি কী	অ্যারে: নিজস্ব কপি, ৬ বাইট	স্ট্রিং লিটারালের পয়েন্টার
অক্ষর বদলানো	s[0] = 'd'; ✓	p[0] = 'd'; ✗ অনির্ধারিত (লিটারাল প্রায়ই রিড-অনলি → ক্র্যাশ)
নতুন টেক্সট দেওয়া	s = "Khulna"; ✗ → strcpy	p = "Khulna"; ✓ নতুন দিকে দেখায়
sizeof	6	8 (পয়েন্টার, 64-বিট)

```
char s[] = "abc";
printf("%zu %zu", sizeof s, strlen(s)); /* 4 3 */
printf("%zu", sizeof("abc"));       /* 4 */
char *p = "Hello";
printf("%c %s", *(p + 1), p + 2);   /* e llo */
printf("%c", "Hello"[4]);           /* o */
char *names[] = {"Ali", "Rina"}; /* 2 ptrs */
```

'A' বনাম "A" 'A' একটি ক্যারেক্টার কনস্ট্যান্ট, C-তে এর টাইপ int: sizeof('A') = 4 (C++-এ 1)। "A" হলো {'A','\0'}: sizeof("A") = 2। char *names[] = পয়েন্টারের অ্যারে; char g[2][5] = নির্দিষ্ট ৫ বাইটের সারি।

মনে রাখার কৌশল "প্রতিটি স্ট্রিং একটি অদৃশ্য দাঁড়ি বয়ে বেড়ায়": দরকারি বাইট = অক্ষর + ১।

পরীক্ষার ফাঁদ s1 == s2 তুলনা করে অ্যাড্রেস, টেক্সট নয় → strcmp। char t[] = {'H','i'};-এ '\0' নেই: %s থামে না।

সমাধানকৃত উদাহরণ ✍️

নিজের strlen

size_t len(const char *s){ const char *p = s; while (*p) p++; return p - s; } → পার্থক্য = দৈর্ঘ্য।

৮ string.h ফাংশন ও স্ট্রিং ইনপুট

প্রায়ই আসে ★

সংজ্ঞা <string.h>-এ আছে '\0'-সমাপ্ত স্ট্রিং ও কাঁচা মেমোরি ব্লকের ফাংশন। কোনোটিই গন্তব্যের আকার জানে না: জায়গা রাখার দায় কলারের, '\0'-এর বাইটসহ। তুলনা হয় **ক্যারেক্টার কোড ধরে অভিধানের ক্রমে (lexicographic)**।

ফাংশন	কাজ	ফেরত / মন্তব্য
strlen(s)	'\0'-এর আগের অক্ষর গোনে	size_t; '\0' গোনা হয় না
strcpy(d, s)	s-কে '\0'সহ d-তে কপি	d; আকার যাচাই নেই
strncpy(d, s, n)	সর্বোচ্চ n অক্ষর কপি	strlen(s) ≥ n হলে **'\0' নেই**
strcat · strncat(d, s, n)	d-এর শেষে s (≤ n অক্ষর) জোড়ে	d; strncat '\0' দেয়
strcmp(a, b) · strncmp	অক্ষর ধরে ধরে তুলনা	**সমান হলে 0**, <0 বা >0
strchr · strrchr · strstr	প্রথম c · শেষ c · সাবস্ট্রিং	পয়েন্টার, নয়তো NULL
strtok · memcpy · memset	ভাগ · কপি · কাঁচা বাইট ভরা	strtok s বদলায়
strrev, strupr, itoa	উল্টানো · বড় হাতের · int → টেক্সট	**স্ট্যান্ডার্ড C নয়**



strcmp অভিধানের মতো কাজ করে, তবে ASCII কোড ধরে

ইনপুট কল	যেখানে থামে	আকার যাচাই	মন্তব্য
scanf("%s", s)	হোয়াইটস্পেসে	নেই → "%49s"	& নেই: s নিজেই অ্যাড্রেস
scanf("%[^\n]", s)	নিউলাইনে	নেই	স্পেসসহ পড়ে
gets(s)	নিউলাইনে	**একেবারে নেই**	**C11-এ বাদ**
fgets(s, n, stdin)	নিউলাইন (রাখে) বা n−1	✓ নিরাপদ	'\0' দেয়; EOF-এ NULL

```
char name[50];
fgets(name, sizeof name, stdin);
name[strcspn(name, "\n")] = '\0';
if (strcmp(name, "Rahim") == 0)
    puts("match");   /* puts adds \n */
char d[6];
strncpy(d, "Bangladesh", 5);
d[5] = '\0';          /* "Bangl" */
```

gets() কেন বাদ আকারের কোনো প্যারামিটার নেই: লম্বা লাইন অ্যারে ছাপিয়ে পাশের মেমোরিতে লেখে — বাফার ওভারফ্লো (Buffer Overflow) (১৯৮৮-র Morris worm fingerd-এ এটাই কাজে লাগিয়েছিল)। C99 (TC3)-এ ডেপ্রিকেটেড, C11-এ বাদ। আরও I/O: পরের লেসন।

মনে রাখার কৌশল "strcmp = ASCII ধরে অভিধানের ক্রম": বড় হাতের (65–90) আসে ছোট হাতের (97–122) আগে।

পরীক্ষার ফাঁদ স্ট্রিং দুটি আলাদা হলে if (strcmp(a, b)) সত্য। নিশ্চিত কেবল চিহ্ন, −1/+1 নয়।

সমাধানকৃত উদাহরণ ✍ মিশ্র ইনপুট

scanf("%d",&n); fgets(s,50,stdin); → fgets পায় বাকি থাকা '\n': একটি **ফাঁকা লাইন**।

পরীক্ষার আগের রাতের পাতা

তথ্য পাতা

<h>: সিস্টেম ডিরেক্টরি · "h": আগে নিজের ফোল্ডার

ডিরেক্টিভ: ; নেই, লাইনের শেষে শেষ

স্ট্রিং বানায় · ## টোকেন জোড়ে

p + n = অ্যাড্রেস + n × sizeof(*p)

arr[i] ≡ *(arr+i) ≡ i[arr]

রো-মেজর: $B + W[N(i-L_r) + (j-L_c)]$

"Hello" = ৬ বাইট; '\0' = ASCII 0

পাইপলাইন: প্রিপ্রসেস → কম্পাইল → অ্যাসেম্বল → লিংক

গার্ড: #ifndef X / #define X / #endif

__FILE__ __LINE__ __DATE__ __TIME__

p2 − p1 = উপাদান, বাইট নয়

রূপান্তর নেই: sizeof arr, &arr

কলাম-মেজর: $B + W[M(j-L_c) + (i-L_r)]$

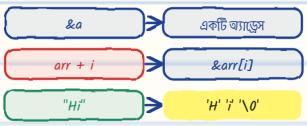
strcmp: সমান 0, <0, >0

➜ MCQ-র জন্য সেরা ১২টি এক-লাইনের কথা

1. printf/scanf → *stdio.h*; malloc → *stdlib.h*
2. conio.h স্ট্যান্ডার্ড C নয়
3. x*x দিলে SQUARE(2+3) → 11
4. const int *p: মান স্থির; int *const p: পয়েন্টার স্থির
5. কাস্ট ছাড়া void * ডিরেফারেন্স হয় না
6. *p++ *p* সরায়; (*p)++ মান বাড়ায়
7. পয়েন্টারের আকার: 4 B (32-বিট), 8 B (64-বিট)
8. &arr + 1 পুরো অ্যারে পেরোয়
9. অ্যারে প্যারামিটার = পয়েন্টার; দৈর্ঘ্য পাঠাও
10. কেবল প্রথম মাত্রা বাদ দেওয়া যায়
11. strlen("abc") = 3, sizeof("abc") = 4
12. gets C11-এ বাদ → fgets

সেরা ফাঁদ ;সহ #define · 100/SQ(5) = 100 · int *x, y; · free-এর পরে ড্যাংলিং · অ্যারে প্যারামিটারের sizeof · কলাম ছাড়া m[][] · char *p = "..." বদলানো · স্ট্রিংয়ে == · '\0' ছাড়া strncpy।

সব কৌশল একসঙ্গে তাক বনাম টেবিল · ভেতরে-বাইরে ব্র্যাকেট · & বলে কোথায়, * বলে কী · টাইপই স্কেল · স্থির বনাম বয়ে বেড়ানো সাইনবোর্ড · রো-মেজর বই পড়ার মতো · অদৃশ্য দাঁড়ি · *strcmp = ASCII অভিধান*।



তিন ধারণা, তিন লাইন

সমাধানকৃত উদাহরণ ✍ এক স্নিপেটে পাঁচ তথ্য char s[] = "Dhaka"; char *p = s + 1; → sizeof s = 6, strlen(s) = 5, *p = 'h', p[2] = 'k', p − s = 1। আর *s = 'd' চলে, কারণ s একটি অ্যারে, লিটারাল নয়।

✓ অনুশীলনী MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

আগে চেষ্টা, পরে মেলাও

১. `#define SQUARE(x) x*x` হলে `SQUARE(3+1)`-এর মান:
ক) 16 খ) 8
গ) 7 ঘ) 10

২. 64-বিট সিস্টেমে `void f(int a[])`-এর ভেতরে `sizeof(a)`:
ক) 8 খ) 20
গ) 5 ঘ) 4

৩. `strlen()` কোন হেডারে ঘোষিত?
ক) *stdio.h* খ) *stdlib.h*
গ) *ctype.h* ঘ) *string.h*

৪. `int *p` (৪ বাইটের *int*)-এ আছে *2000*। `p + 3`:
ক) 2003 খ) 2012
গ) 2006 ঘ) 2024

৫. `const int *p;`-এর অর্থ:
ক) `*p` রিড-অনলি খ) `p` রিড-অনলি
গ) দুটোই স্থির ঘ) কোনোটিই স্থির নয়

৬. `int A[5][4]`, বেস *1000*, ৪ বাইটের *int*, রো-মেজর। `A[3][2]`-এর অ্যাড্রেস:
ক) 1052 খ) 1048
গ) 1056 ঘ) 1060

৭. C11 স্ট্যান্ডার্ড থেকে কোন ফাংশন বাদ দেওয়া হয়েছে?
ক) *fgets* খ) *gets*
গ) *puts* ঘ) *scanf*

৮. `char s[] = "Dhaka";` — `sizeof(s)`:
ক) 5 খ) 4
গ) 8 ঘ) 6

৯. `strcmp("abc", "abd")` ফেরত দেয়:
ক) 0 খ) ধনাত্মক মান
গ) ঋণাত্মক মান ঘ) 3

১০. একটি হেডার দুবার যুক্ত হওয়া আটকায়:
ক) `#pragma pack` খ) `#ifndef` / `#define` / `#endif`
গ) `#error` ঘ) `#undef`

উত্তরমালা → ১-গ · ২-ক · ৩-ঘ · ৪-খ · ৫-ক · ৬-গ · ৭-খ · ৮-ঘ · ৯-গ · ১০-খ

লিখিত প্রশ্ন C-তে অ্যারে ও পয়েন্টারের সম্পর্ক ব্যাখ্যা করো। `arr[i]` কীভাবে মূল্যায়িত হয় দেখাও, অ্যারে ও পয়েন্টারে `sizeof` কেন ভিন্ন ফল দেয় বলো, `A[i][j]`-এর রো-মেজর অ্যাড্রেস নির্ণয় করো, এবং পয়েন্টার দিয়ে স্ট্রিংয়ের দৈর্ঘ্য ফেরত দেয় এমন একটি ফাংশন লেখো। ১০ নম্বর

১ **রূপান্তর (২)** — এক্সপ্রেশনে অ্যারের নাম হয়ে যায় `&arr[0]`; `arr[i]`-এর সংজ্ঞাই `*(arr + i)`, আর `arr + i` যোগ করে `i × sizeof(int)` বাইট। ২

২ ***sizeof* (২)** — `sizeof`/`&`-এর সঙ্গে রূপান্তর হয় না: `int arr[5]` → *20*; পয়েন্টার (বা অ্যারে প্যারামিটার) → 64-বিটে *8*; নামটিতে অ্যাসাইন বা ইনক্রিমেন্ট চলে না। ২

৩ **অ্যাড্রেস (৩)** — সারিগুলো পরপর থাকে: N উপাদানের i টি পুরো সারি পেরিয়ে তারপর j: $LOC = B + W[N(i - L_r) + (j - L_c)]$; যেমন `int A[5][4]` *1000*-এ → `A[3][2]` = *1056*। ৩

৪ **ফাংশন (৩)** — `size_t len(const char *s){ const char *p = s; while (*p) p++; return p - s; }`: `'\0'` পর্যন্ত এগোও, পয়েন্টারের পার্থক্যই দৈর্ঘ্য। ৩

→ পরীক্ষার আগে ✓

- ✓ যেকোনো সাধারণ ফাংশনের হেডার বলতে পারি
- ✓ ম্যাক্রো প্রসারিত করে ফাঁদ ধরতে পারি
- ✓ `p + n` ও `p2 - p1` হিসাব করতে পারি
- ✓ `arr`, `&arr`, `sizeof` আলাদা করতে পারি
- ✓ দুটি 2D অ্যাড্রেস সূত্রই খাটাতে পারি
- ✓ `'\0'`সহ স্ট্রিংয়ের বাইট গুনতে পারি